# Python OOP Part 2 - Beginner Notes

## 1. How Object Access Attributes

**Objects access attributes using `__dict__` (dictionary) internally.**

```
class Student:
def init(self, name, age):
self.name = name
self.age = age

student1 = Student("Alice", 20)

# Access attributes normally

print(student1.name) # Output: Alice
print(student1.age) # Output: 20

# Access using dict

print(student1.dict) # Output: {'name': 'Alice', 'age': 20}

# Access specific attribute via dict

print(student1.dict['name']) # Output: Alice
```

**Key Points:**

- Python stores object attributes in `__dict__` dictionary
- `__dict__` contains key-value pairs (attribute_name: value)
- Can access, modify, add, delete attributes using `__dict__`

**Example 2: Using `__dict__` to check all attributes**

```
class Car:
def init(self, brand, model, year):
self.brand = brand
self.model = model
self.year = year

car1 = Car("Toyota", "Camry", 2023)
```

# Display all attributes

for key, value in car1.**dict**.items():
print(f"{key}: {value}")

# Output:

# brand: Toyota

# model: Camry

# year: 2023

## 2. Attribute Creation from Outside of the Class

**You can add new attributes to an object from outside the class.**

class Student:
def **init**(self, name):
self.name = name

student1 = Student("Alice")
print(student1.**dict**) # Output: {'name': 'Alice'}

# Add new attribute from outside

student1.age = 20
student1.city = "Mumbai"

print(student1.**dict**) # Output: {'name': 'Alice', 'age': 20, 'city': 'Mumbai'}
print(student1.age) # Output: 20
print(student1.city) # Output: Mumbai

**Using `__dict__` to add attributes:**

student2 = Student("Bob")
student2.**dict**['grade'] = 'A'
student2.**dict**['marks'] = 85

print(student2.**dict**) # Output: {'name': 'Bob', 'grade': 'A', 'marks': 85}

**Key Points:**

- Attributes can be added dynamically at runtime
- Each object has its own `__dict__`
- New attributes don't affect the class or other objects

## 3. Reference Variables

**Reference variables store the memory address (reference) of an object, not the object itself.**

```
class Student:
def __init__(self, name):
self.name = name

# student1 holds reference to object

student1 = Student("Alice")
print(student1.name) # Output: Alice

# student2 references the SAME object

student2 = student1
print(student2.name) # Output: Alice

# Modify through student2

student2.name = "Bob"

# Both variables reflect the change (same object)

print(student1.name) # Output: Bob
print(student2.name) # Output: Bob
```

**Check if references point to same object:**

```
print(student1 is student2) # Output: True
print(id(student1)) # Same memory address
print(id(student2)) # Same memory address
```

**Creating independent objects:**

```
student3 = Student("Charlie")
student4 = Student("David")

print(student3 is student4) # Output: False
print(id(student3)) # Different address
print(id(student4)) # Different address
```

## 4. Pass by Reference

**In Python, objects are passed by reference to functions.**

```
class Student:
def init(self, name, marks):
self.name = name
self.marks = marks

def update_marks(student, new_marks):
student.marks = new_marks # Modifies original object

student1 = Student("Alice", 75)
print("Before:", student1.marks) # Output: Before: 75

update_marks(student1, 90)
print("After:", student1.marks) # Output: After: 90
```

**Passing multiple objects:**

```
def swap_names(s1, s2):
s1.name, s2.name = s2.name, s1.name

student1 = Student("Alice", 80)
student2 = Student("Bob", 85)

print("Before swap:", student1.name, student2.name) # Output: Before swap: Alice Bob
swap_names(student1, student2)
print("After swap:", student1.name, student2.name) # Output: After swap: Bob Alice
```

**Key Point:** When you pass an object to a function, you're passing the reference, not a copy.

## 5. Mutability of Object

**Mutable objects can be changed after creation. Immutable objects cannot.**

### Mutable Objects (can be changed)

```
class Student:
def init(self, name, marks):
self.name = name
self.marks = marks

# Student object is mutable

student1 = Student("Alice", 75)
print(student1.marks) # Output: 75

student1.marks = 90 # Can modify
print(student1.marks) # Output: 90

student1.city = "Mumbai" # Can add new attributes
print(student1.city) # Output: Mumbai
```

**Lists, Dictionaries, Sets are mutable:**

```
my_list = [1, 2, 3]
my_list.append(4) # Modifies original list
print(my_list) # Output: [1, 2, 3, 4]
```

### Immutable Objects (cannot be changed)

# Strings, Integers, Tuples are immutable

```
x = 10
print(id(x)) # Memory address

x = 20 # Creates NEW object, x references new location
print(id(x)) # Different memory address

name = "Alice"
print(id(name))

name = "Bob" # Creates NEW string
print(id(name)) # Different address
```

**Key Points:**

- **Mutable:** Lists, Dictionaries, Sets, Custom Objects
- **Immutable:** Integers, Floats, Strings, Tuples
- Modifying immutable types creates new objects

## 6. Encapsulation

**Encapsulation means hiding internal details and restricting direct access to data.**

**Access Modifiers:**

- **Public:** Accessible everywhere (no prefix)
- **Protected:** Accessible within class and subclasses (single underscore _)
- **Private:** Accessible only within class (double underscore __)

### a) Public Attributes

```
class Student:
def init(self, name, age):
self.name = name # Public
self.age = age # Public

student1 = Student("Alice", 20)
print(student1.name) # Output: Alice (accessible)
```

### b) Protected Attributes (Convention)

class Student:
def **init**(self, name, age):
self.name = name
self._age = age # Protected (single underscore)

student1 = Student("Alice", 20)
print(student1._age) # Output: 20 (accessible but shouldn't use directly)

### c) Private Attributes (Name Mangling)

class Student:
def **init**(self, name, age):
self.name = name
self.__age = age # Private (double underscore)

```
def get_age(self):
    return self.__age  # Access private attribute inside class

def set_age(self, age):
    if age > 0:
        self.__age = age
```

student1 = Student("Alice", 20)

# Cannot access directly

# print(student1.__age) # Error: AttributeError

# Access using public method

print(student1.get_age()) # Output: 20

# Modify using setter

student1.set_age(21)
print(student1.get_age()) # Output: 21

**Name Mangling:** Python internally renames `__age` to `_Student__age`

print(student1._Student__age) # Output: 21 (don't do this!)

**Complete Encapsulation Example:**

class BankAccount:
def **init**(self, account_number, balance):
self.__account_number = account_number # Private
self.__balance = balance # Private

```
def deposit(self, amount):
    if amount > 0:
        self.__balance += amount
        print(f"Deposited: {amount}")

def withdraw(self, amount):
    if 0 < amount <= self.__balance:
        self.__balance -= amount
        print(f"Withdrawn: {amount}")
    else:
        print("Insufficient balance")

def get_balance(self):
    return self.__balance
```

account = BankAccount("123456", 5000)
account.deposit(2000) # Output: Deposited: 2000
account.withdraw(1500) # Output: Withdrawn: 1500
print(account.get_balance()) # Output: 5500

# Cannot access private attributes directly

# print(account.__balance) # Error

## 7. Collection of Objects

**You can store multiple objects in lists, dictionaries, or other collections.**

### a) List of Objects

class Student:
def **init**(self, name, marks):
self.name = name
self.marks = marks

```
def display(self):
```

```
    print(f"Name: {self.name}, Marks: {self.marks}")
```

# Create list of objects

students = [
Student("Alice", 85),
Student("Bob", 90),
Student("Charlie", 78)
]

# Access objects from list

for student in students:
student.display()

# Output:

# Name: Alice, Marks: 85

# Name: Bob, Marks: 90

# Name: Charlie, Marks: 78

### b) Dictionary of Objects

class Employee:
def **init**(self, name, salary):
self.name = name
self.salary = salary

# Dictionary with employee ID as key

employees = {
101: Employee("Alice", 50000),
102: Employee("Bob", 60000),
103: Employee("Charlie", 55000)
}

# Access objects from dictionary

print(employees[101].name) # Output: Alice
print(employees[102].salary) # Output: 60000

# Loop through dictionary

for emp_id, emp_obj in employees.items():
print(f"ID: {emp_id}, Name: {emp_obj.name}, Salary: {emp_obj.salary}")

### c) Finding Object with Specific Property

students = [
Student("Alice", 85),
Student("Bob", 90),
Student("Charlie", 78)
]

# Find student with highest marks

top_student = max(students, key=lambda s: s.marks)
print(f"Top Student: {top_student.name} with {top_student.marks} marks")

# Output: Top Student: Bob with 90 marks

## 8. Static Variables

**Static variables (class variables) are shared among ALL objects of a class.**

class Student:
school_name = "ABC School" # Static variable (shared by all)

```
def __init__(self, name, roll_no):
    self.name = name        # Instance variable (unique to each object)
    self.roll_no = roll_no
```

# Access static variable using class name

print(Student.school_name) # Output: ABC School

# Create objects

student1 = Student("Alice", 101)
student2 = Student("Bob", 102)

# All objects share the same static variable

print(student1.school_name) # Output: ABC School
print(student2.school_name) # Output: ABC School

# Modify static variable using class name

Student.school_name = "XYZ School"

print(student1.school_name) # Output: XYZ School
print(student2.school_name) # Output: XYZ School

**Counting Objects using Static Variable:**

class Student:
count = 0 # Static variable to count objects

```
def __init__(self, name):
    self.name = name
    Student.count += 1  # Increment count when object created
```

student1 = Student("Alice")
student2 = Student("Bob")
student3 = Student("Charlie")

print(f"Total students: {Student.count}") # Output: Total students: 3

**Key Points:**

- Static variables are defined outside `__init__`
- Shared by all objects
- Access using class name (recommended) or object name
- Modifying via class name affects all objects

## 9. Static Methods

**Static methods belong to the class, not to any specific object. They don't use `self` or `cls`.**

**Use `@staticmethod` decorator to create static methods.**

class Calculator:
@staticmethod

def add(a, b):
return a + b

```
@staticmethod
def multiply(a, b):
    return a * b
```

# Call static method using class name (no object needed)

print(Calculator.add(10, 20)) # Output: 30
print(Calculator.multiply(5, 6)) # Output: 30

# Can also call using object (not recommended)

calc = Calculator()
print(calc.add(15, 25)) # Output: 40

**Practical Example: Utility Class**

class MathUtils:
@staticmethod
def is_even(num):
return num % 2 == 0

```
@staticmethod
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True

@staticmethod
def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
```

```
    for i in range(2, n + 1):
        result *= i
    return result
```

# Use static methods without creating object

print(MathUtils.is_even(10)) # Output: True
print(MathUtils.is_prime(7)) # Output: True
print(MathUtils.factorial(5)) # Output: 120

**Static Method vs Instance Method:**

class Student:
school_name = "ABC School" # Static variable

```
    def __init__(self, name):
        self.name = name

    # Instance method (uses self)
    def display(self):
        print(f"Student: {self.name}")

    # Static method (no self, no cls)
    @staticmethod
    def get_school_info():
        return "ABC School is located in Mumbai"
```

student1 = Student("Alice")
student1.display() # Instance method needs object

print(Student.get_school_info()) # Static method - no object needed

# Output: ABC School is located in Mumbai

**When to Use Static Methods:**

- Utility functions related to class
- Methods that don't need access to instance or class data
- Helper functions logically grouped with class
- Functions that don't modify object state

**Difference Table:**

| Feature | Instance Method | Static Method |
|---|---|---|
| Decorator | None | @staticmethod |
| First parameter | self (required) | No self or cls |
| Access instance data | Yes | No |
| Access class data | Yes | No |
| Called using | Object | Class or Object |
| Use case | Object-specific operations | Utility functions |

Table 1: Instance Method vs Static Method

---

## Quick Reference

| Concept | Example |
|---|---|
| Access attributes | obj.__dict__ |
| Add attribute outside | obj.new_attr = value |
| Reference variable | var2 = var1 |
| Pass by reference | Object passed to function |
| Public attribute | self.name |
| Protected attribute | self._age |
| Private attribute | self.__salary |
| Static variable | ClassName.var = value |
| Static method | @staticmethod def func(): |

Table 2: OOP Part 2 Quick Reference

| Access Level | Syntax |
|---|---|
| Public | self.variable |
| Protected | self._variable |
| Private | self.__variable |

Table 3: Encapsulation Access Levels